Projet « Moteur de recherche » — Partie 3 sur 3

Interface graphique et extensions

Programmation de spécialité (Python) (≈ 5–6 h)

Julien Velcin* Walid Bechkit†

Année universitaire 2026–2027

Cette **dernière partie** rend votre moteur *utilisable* par une personne qui n'est pas informaticienne. Jusqu'ici, on interrogeait le corpus en écrivant du code ; on va maintenant construire une **interface graphique conviviale**. L'objectif est de permettre d'explorer facilement la masse de documents que vous avez constituée (par ex. pour des chercheurs en sciences humaines : sociologues, linguistes…).

On **réutilise tout l'existant** : les classes `Document`, `Corpus` et `SearchEngine` des Parties 1 et 2, *sans* rien reconstruire. On travaille directement sur **votre corpus** (Reddit + Arxiv) sur votre thématique. Cette partie correspond à la version **v3** du rendu : le **cœur guidé** (Parties 1 et 2 ci-dessous) est **obligatoire** ; les **extensions** (Partie 3) sont des *bonus au choix*, qui viennent **en plus** et ne sont pas exigées.

Partie 1 : Préparation : réutiliser le moteur dans un notebook

L'interface vivra dans un **carnet de notes** (*notebook* Jupyter). Conformément aux consignes du projet, seule *l'interface* est réalisée dans le notebook ; le reste du code reste dans vos modules `.py`.

1.1 Dans un notebook, **importez** vos classes (`Corpus`, `Document`, `SearchEngine`) depuis vos modules, puis **chargez** votre corpus déjà sauvegardé (méthode `load`). Il ne faut **pas** réinterroger les API Reddit / Arxiv : on repart du fichier produit en Partie 1.

1.2 Instanciez votre moteur : `moteur = SearchEngine(corpus)`. Vérifiez rapidement, par deux ou trois requêtes en mots-clés, que la recherche fonctionne (rappel de la Partie 2).

1.3 Sur un gros corpus, la construction de la matrice ou la boucle de recherche peut être longue. Ajoutez un **compteur de progression** à l'aide de la librairie `tqdm` (cf. https://tqdm.github.io) afin de visualiser l'avancement de la boucle.

*Auteur du sujet original.

†Responsable du cours

Partie 2 : Petite interface graphique avec `ipywidgets`

On construit une interface simple à base de *widgets* intégrés au notebook, avec la librairie `ipywidgets`. N'hésitez pas à consulter la documentation (https://ipywidgets.readthedocs.io/en/stable/examples/Widget%20Basics.html) et des exemples en ligne.

Interface visée (de haut en bas) :

- un **titre** « Moteur de recherche » ;
- un champ **« Mots clés »** où saisir la requête ;
- un curseur **« Nombre d'articles à extraire »** ;
- un bouton **« Rechercher »** ;
- une **zone de résultats** affichant les documents trouvés.

2.1 Créez d'abord les objets graphiques simples : un `Label` (le titre), un `Text` (pour recueillir les mots-clés) et un `IntSlider` (pour le nombre de documents à retourner).

2.2 Pour afficher ces éléments, on les empile avec des conteneurs `VBox` (vertical) et `HBox` (horizontal), passés à la fonction `display`, par exemple : `display(widgets.VBox([...]))` où `[...]` est la *liste des widgets* à disposer.

2.3 Créez un objet `Output` qui recevra la sortie de la recherche, et ajoutez-le à l'interface précédente.

2.4 Ajoutez un objet `Button` (« Rechercher ») qui déclenche une fonction personnalisée `clique_bouton`. Le branchement se fait via la méthode `on_click` du bouton.

2.5 Écrivez la fonction `clique_bouton` qui lance la recherche à partir des widgets : elle récupère les mots-clés (attribut `.value` du champ texte) et le nombre de documents (valeur du curseur), appelle la méthode `search` de votre moteur, et affiche le résultat **dans** l'objet `Output` grâce à l'instruction `with` (https://ipywidgets.readthedocs.io/en/stable/examples/Output%20Widget.html). Pensez à **vider** la zone de sortie à chaque appel avec `clear_output()`.

2.6 Testez l'interface en faisant varier les mots-clés et le nombre de documents retournés. Vérifiez que les résultats correspondent bien à ceux obtenus en ligne de commande en Partie 2.

Partie 3 : Extensions (bonus au choix, non obligatoires)

Une fois le cœur ci-dessus fonctionnel, vous pouvez enrichir votre outil. Ces extensions sont **facultatives** et viennent **en plus** : choisissez celles qui vous intéressent, selon le temps et vos envies. Inutile de toutes les faire ; une extension *bien réalisée* vaut mieux que plusieurs ébauches. Chacune est valorisée dans la note (« améliorations du projet »).

Bonus 1 — Filtres de recherche Enrichissez l'interface pour filtrer les résultats, par exemple par **source** (Reddit / Arxiv, via `getType`), par **auteur**, ou par **date / période**. Côté widgets : un `Dropdown` pour la source ou l'auteur, un `SelectionRangeSlider` ou deux `DatePicker` pour la période.

Bonus 2 — Interface web avec Streamlit À la place (ou en complément) d'`ipywidgets`, proposez une véritable petite **application web** avec **Streamlit** (https://streamlit.io) : un champ `st.text_input` pour la requête, un `st.slider` pour le nombre de résultats, un `st.button` « Rechercher » et un `st.dataframe` pour l'affichage. L'application se lance par `streamlit run app.py`. C'est l'occasion de découvrir un outil très utilisé en science des données.

Bonus 3 — Enrichir le corpus (nouvelles sources) Ajoutez une **nouvelle source** de documents : un autre subreddit, une nouvelle requête Arxiv, un **flux RSS** d'actualités, un export CSV, etc. Grâce à l'**héritage** et au **patron d'usine** de la Partie 1, l'ajout d'une source revient à écrire une nouvelle classe fille de `Document` et à l'enregistrer dans la fabrique — sans toucher au reste.

Bonus 4 — Analyse comparative de deux sous-corpus Proposez une **comparaison** entre deux sous-corpus (par ex. les documents *Reddit* contre ceux d'*Arxiv*) : quels sont les **mots communs** ? les **mots spécifiques** à chacun ? Une approche simple consiste à traiter chaque sous-corpus comme un **« gros document »** et à comparer l'importance des mots (scores TF–IDF) d'un côté et de l'autre.

Bonus 5 — Évolution temporelle d'un mot Suivez l'**importance d'un mot** (ou d'un groupe de mots) **au cours du temps**. Il faut exploiter le champ `date` pour découper la frise temporelle en **périodes**, calculer la fréquence (ou le score TF–IDF) par période, puis tracer une courbe d'évolution.

Bonus 6 — Une autre mesure d'importance : Okapi BM25 En plus de TF–IDF, implémentez la mesure **Okapi BM25** (https://fr.wikipedia.org/wiki/Okapi_BM25), un raffinement très utilisé en recherche d'information. Comparez les classements obtenus par les deux mesures sur quelques requêtes.

Bonus 7 — Visualisations Ajoutez des **figures** pour rendre l'exploration plus parlante : un diagramme en **barres** des mots les plus importants, une **courbe** d'évolution temporelle, ou un **nuage de mots** (`wordcloud`). Ces graphiques (matplotlib) peuvent s'afficher directement dans la zone `Output` de votre interface.

Bonus 8 — Concordancier interactif Intégrez à l'interface la fonction `concorde` de la Partie 2 : pour un mot de la requête, affichez les passages trouvés avec leur contexte gauche / droit, sous forme de tableau.

Toutes les (bonnes) idées sont les bienvenues ! N'hésitez pas à tester les librairies Python qui vous intéressent et à proposer vos propres pistes.